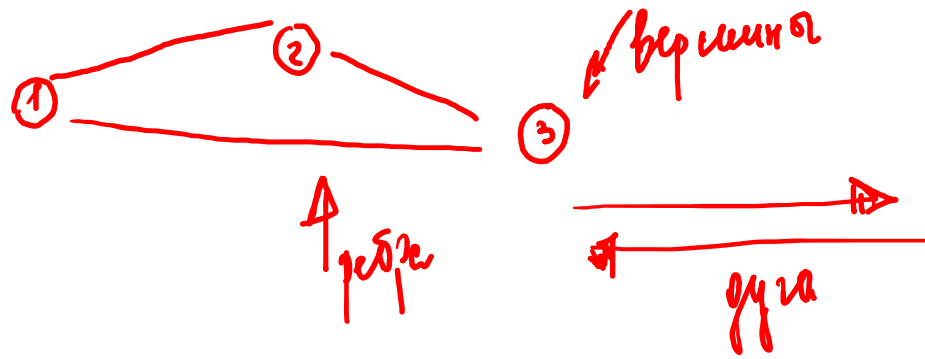
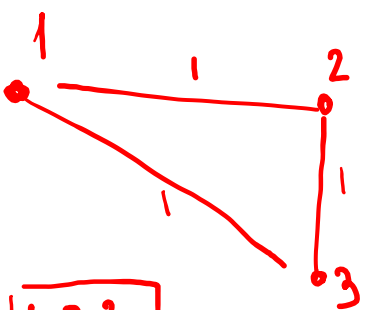




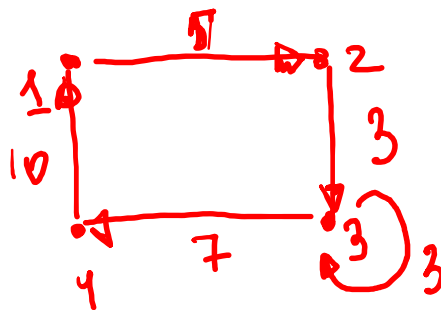
Курс «Комбинаторика и
теория графов»
Лекция 3. Теория графов.
Введение. Обходы.



1: 2, 3
2: 1, 3
3: 1, 2

	1	2	3
1	0	1	1
2	1	0	1
3	1	1	0

1 2
2 3
3 1



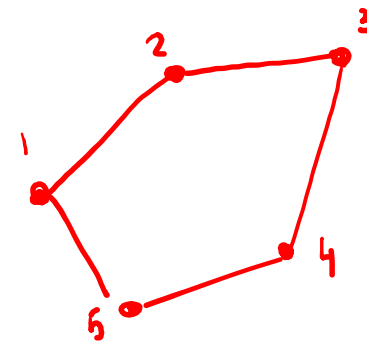
1: (2, 5)
2: (3, 3)
3: (4, 7) (3, 3) ...

	1	2	3	4
1	0	5	0	0
2	0	0	3	0
3	0	0	3	7
4	10	0	0	0

- Матрица смежности представляет собой таблицу, в которой номера столбцов и строк означают номера вершин графа. На пересечении строк и столбцов ставится 1, если вершины соединены ребром в графе, и 0, если не соединены.
- В матрице инцидентий номера строк – номера вершин, а номера столбцов – номера ребер (дуг). На их пересечении ставится 1, если ребро и вершина инцидентны (т. е. ребро соединено с данной вершиной).
- Список смежности. Каждой вершине графа соответствует список, состоящий из «соседей» этой вершины.
- Список дуг (массив дуг/ребер). В первой строке которого хранится информация, из какой вершины начинается дуга, во второй - в какой кончается, а в третьей строке - вес дуги.

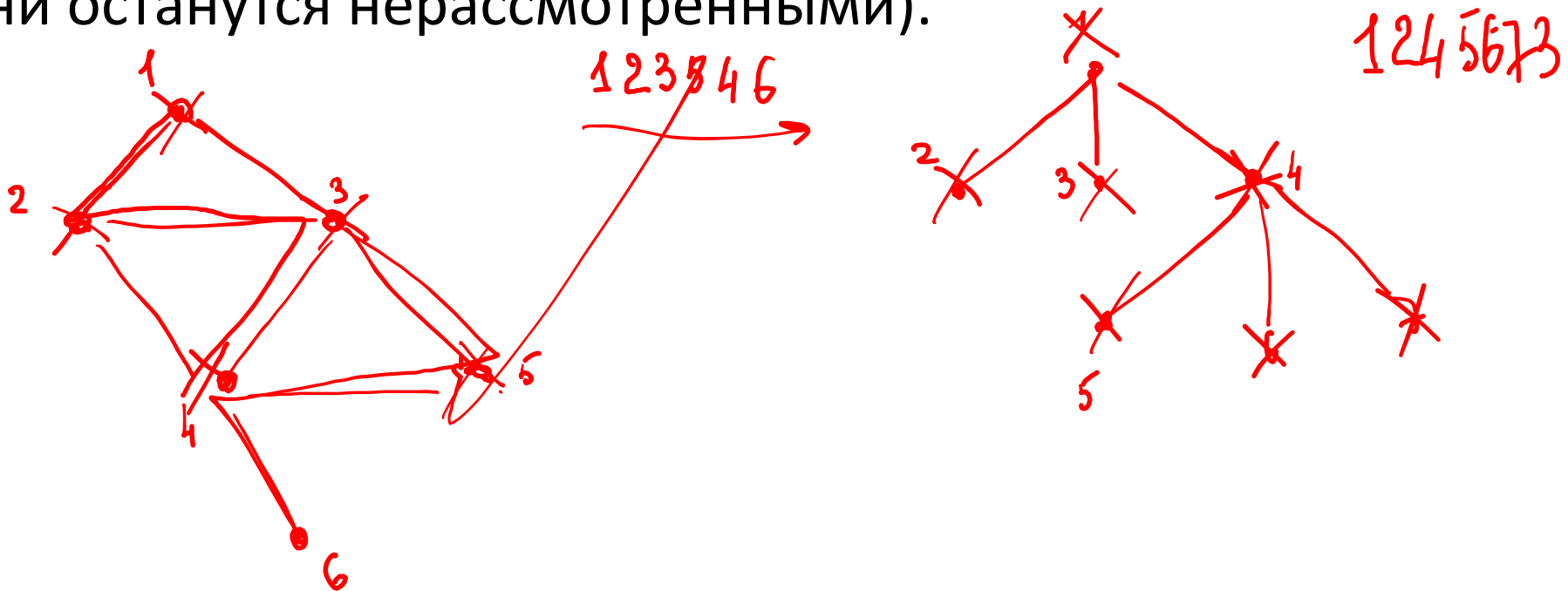
Обход графа (поиск на графах)

- Под **обходом графов (поиском на графах)** понимается процесс *систематического* просмотра всех ребер или **вершин графа** с целью отыскания ребер или вершин, удовлетворяющих некоторому условию.
- При решении многих задач, использующих графы, необходимы эффективные методы регулярного обхода вершин и ребер графов. К стандартным и наиболее распространенным методам относятся:
 - *поиск в глубину* (Depth First Search, *DFS*);
 - *поиск в ширину* (Breadth First Search, *BFS*).

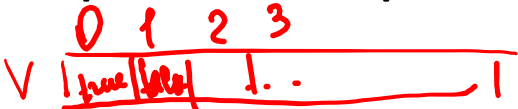


Поиск в глубину

Идея поиска в глубину – когда возможные пути по ребрам, выходящим из вершин, разветвляются, нужно сначала полностью исследовать одну ветку и только потом переходить к другим веткам (если они останутся нерассмотренными).



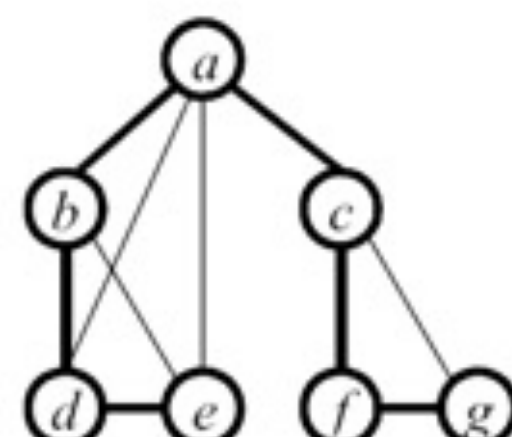
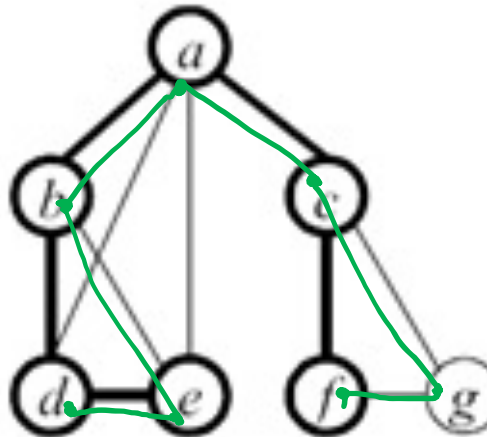
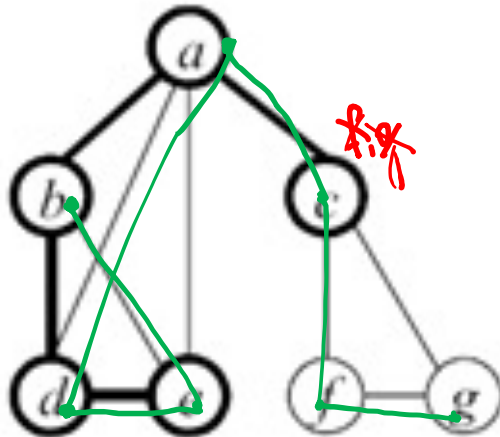
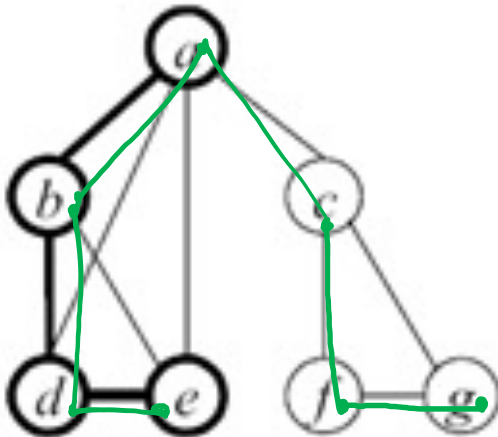
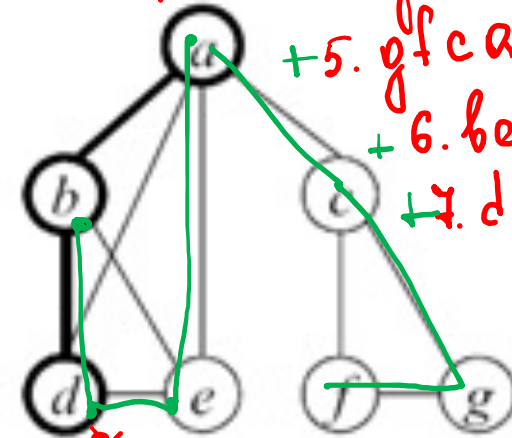
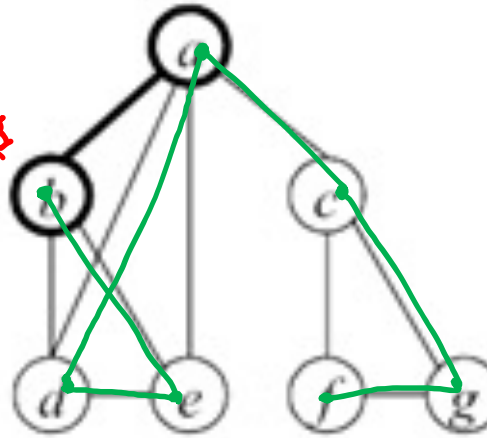
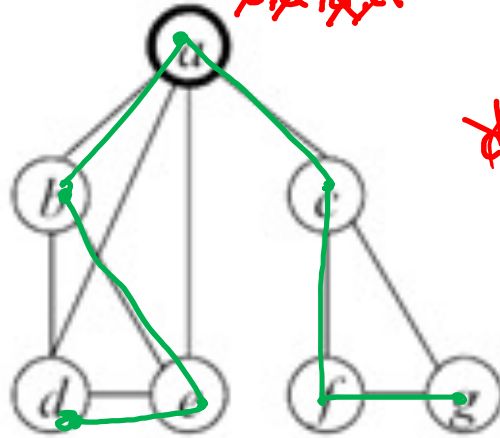
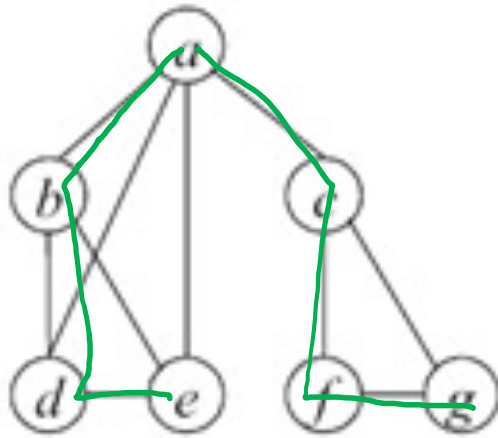
Алгоритм поиска в глубину

- **Шаг 1.** Всем вершинам графа присваивается *значение* не посещенная. Выбирается первая *вершина* и помечается как посещенная.

- **Шаг 2.** Для последней помеченной как посещенная вершины выбирается смежная *вершина*, являющаяся первой помеченной как не посещенная, и ей присваивается *значение* посещенная. Если таких вершин нет, то берется предыдущая помеченная *вершина*.
- **Шаг 3.** Повторить шаг 2 до тех пор, пока все вершины не будут помечены как посещенные.

~~a b c d e f g~~
~~1 1 1 1 1 1 1~~
 a b c d e f g

Пример

- + 1. a c f g b d e
- + 2. a b e d c f g
- + 3. a d e b c g f
- + 4. a e d b c g f
- + 5. g f c a b d e
- + 6. b e d a c f g
- + 7. d e b a c g f

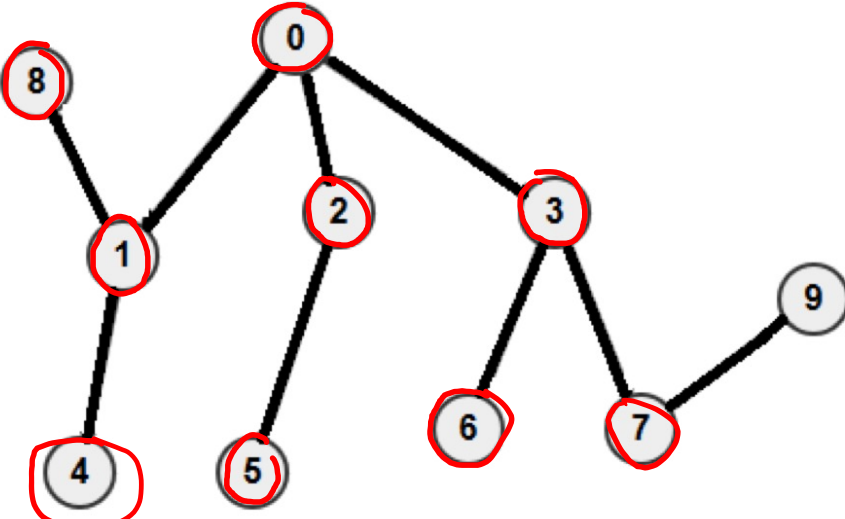


Рекурсивная реализация

used	0	1	2	3	4	5	6	7	8	9
	1	1	1	1	1	1	1	1	1	1

0	1	2	3	4	5	6	7	8	9
1	1	1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4	4	4
5	5	5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8	8
9	9	9	9	9	9	9	9	9	9

0 1 4 8 2 5 3 6 7 9

Неориентированный граф G	Функция обхода графа в глубину DFS
	<pre>void dfs(int v) { used[v]=1; for (auto i : g[v]) → 2E if (!used[i]) dfs(i); → V }</pre> Вызов из основного текста программы → dfs (0)

Программа обойдет вершины графа в следующем порядке: 0, 1, 4, 8, 2, 5, 3, 6, 7, 9.

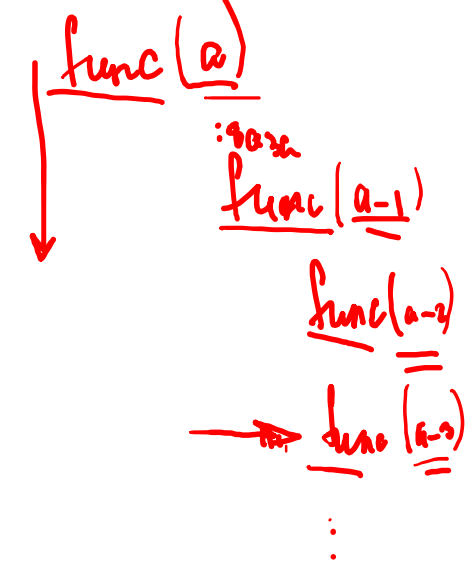
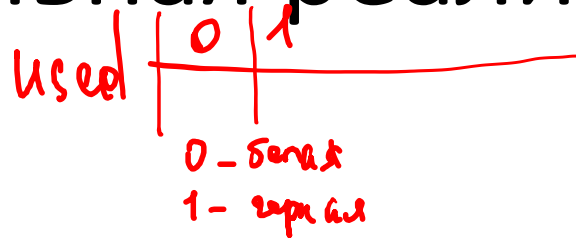
Оценка сложности алгоритма включает в себя следующие операции:

- просмотр всех $|V|$ вершин, для каждой из которых v просматриваются ее соседи;
- просмотр всех соседей вершины v . При этом алгоритм проходит по ребру $\{v, u\}$. Причем, каждое такое ребро $\{v, u\}$ просматривается дважды: для вершины u и для вершины v .

Итоговая сложность алгоритма `dfs`, таким образом $O(|V| + |E|)$.

Не рекурсивная реализация

$a=1000$

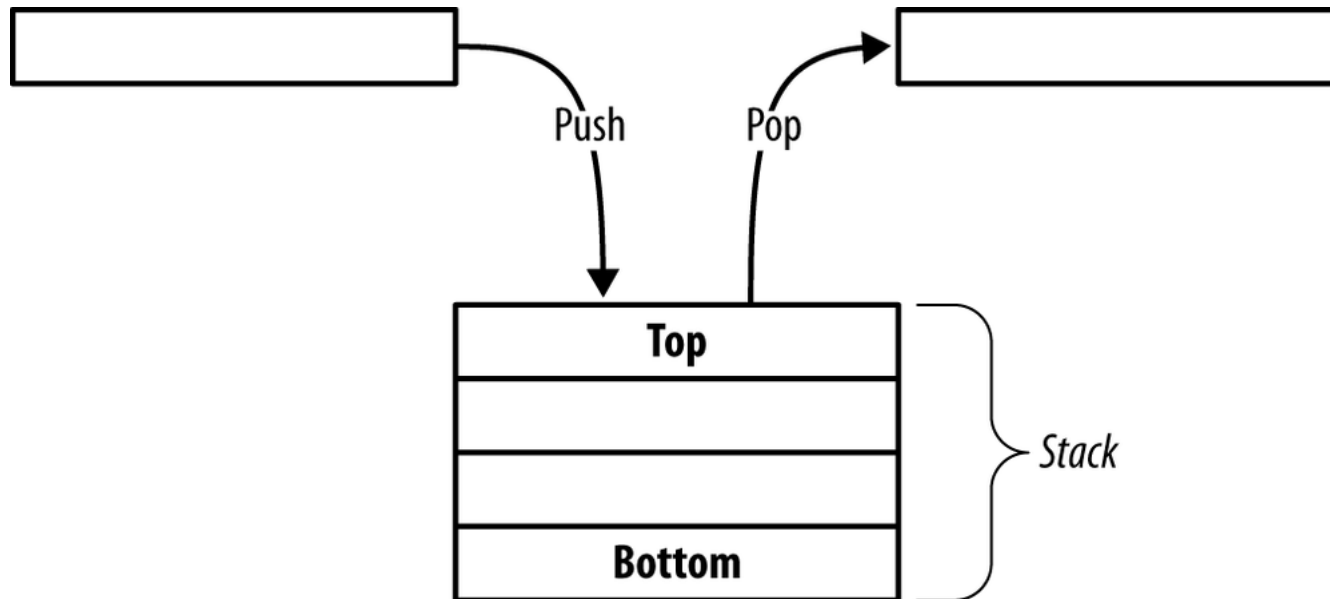


- Все вершины белые
- Выбираем $v \in V$
- Помечаем v черным и помещаем в некоторый контейнер T (стек)
- Пока T не пусто:
 - Извлекаем v из T
 - Производим некоторые действия над v (зависит от задачи)
 - $\forall v' \in \Gamma(v)$:
 - Если v' - белая, то помечаем v' черным и помещаем в T



Stack

- **Стек** — это структура данных, которая работает по принципу **FILO** (first in — last out; первый пришел — последний ушел) или (*Last-In-First-Out* или **LIFO**).



Операции в стеке

- создание стека;
- печать (просмотр) стека;
- добавление элемента в вершину стека (*push*);
- извлечение элемента из вершины стека (*pop*);
- проверка пустоты стека;
- очистка стека.

В C++ уже есть готовый шаблон — `stack`

Stack: empty

Push 1

Stack: 1

Push 2

Stack: 1 2

Push 3

Stack: 1 2 3

Push 4

Stack: 1 2 3 4

Pop

Stack: 1 2 3

Pop

Stack: 1 2

Pop

Stack: 1

<stack>

- Создание stack <тип данных> <имя>;
- Добавить элемент push(); $O(1)$
- Проверить на пустоту empty(); $O(1)$
- Удалить верхний элемент pop(); $O(1)$
- Посмотреть верхний элемент top(); $O(1)$
- Размер стека size(). $O(1)$

Stack C#

```
using System;
using System.Collections.Generic;
namespace Collections
{
    class Program
    {
        static void Main(string[] args)
        {
            Stack<int> numbers = new Stack<int>();
            numbers.Push(3); // в стеке 3
            numbers.Push(5); // в стеке 5, 3
            numbers.Push(8); // в стеке 8, 5, 3
            int stackElement = numbers.Pop();
            Console.WriteLine(stackElement);
        }
    }
}
```

Применение

Поиск в глубину — хороший инструмент для исследования топологических свойств графов. Например:

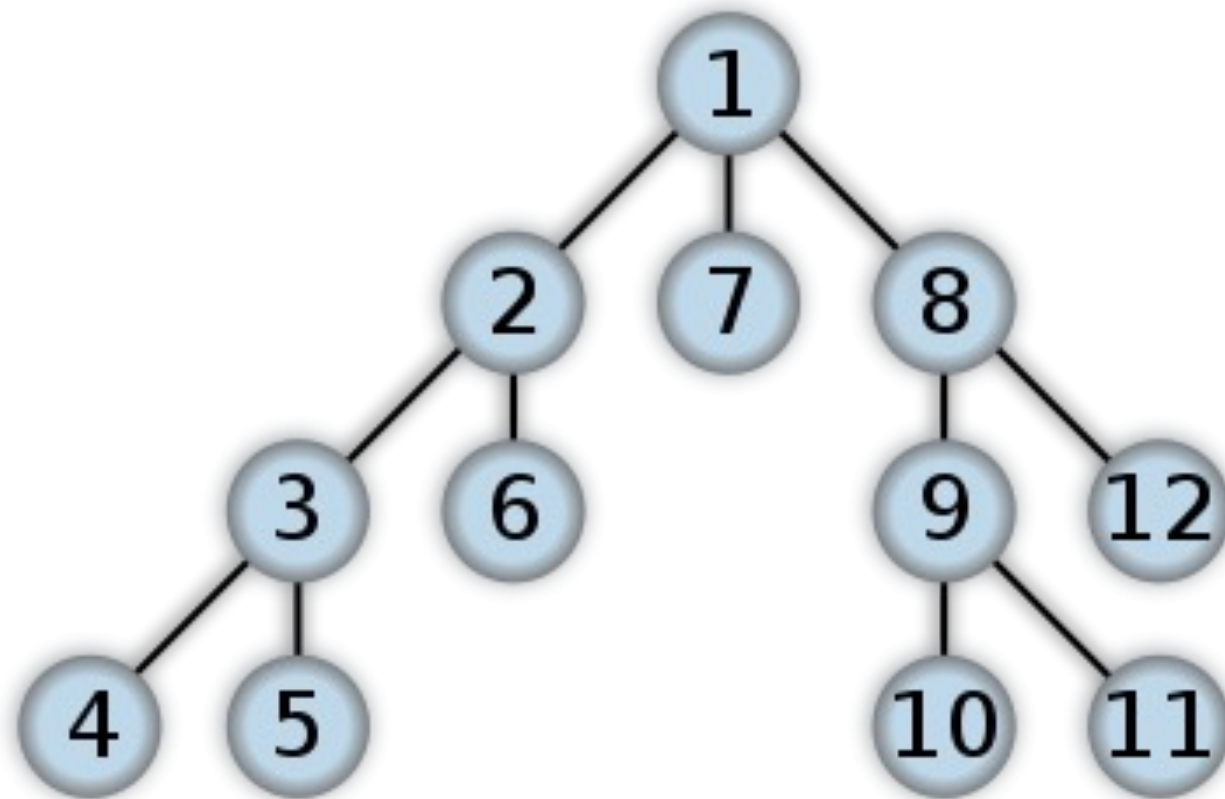
- В качестве подпрограммы в алгоритмах поиска одно- и двусвязных компонент.
- В топологической сортировке.
- Для поиска точек сочленения, мостов.
- В различных расчётах на графах. Например, как часть алгоритма Диница поиска максимального потока.

Реализации

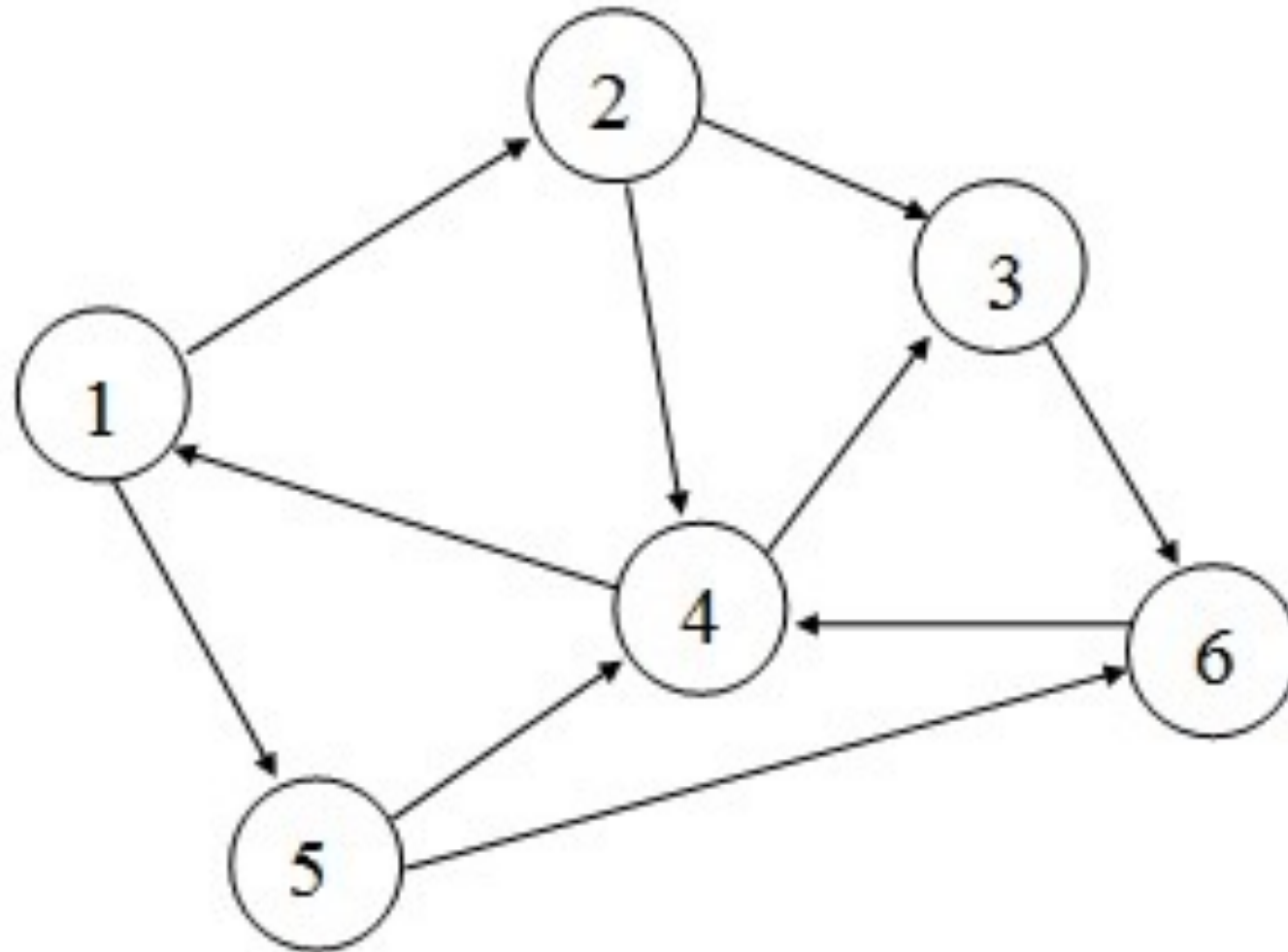
- [https://ru.wikibooks.org/wiki/Реализации алгоритмов/Поиск в глубину](https://ru.wikibooks.org/wiki/Реализации_алгоритмов/Поиск_в_глубину)

// ссылка

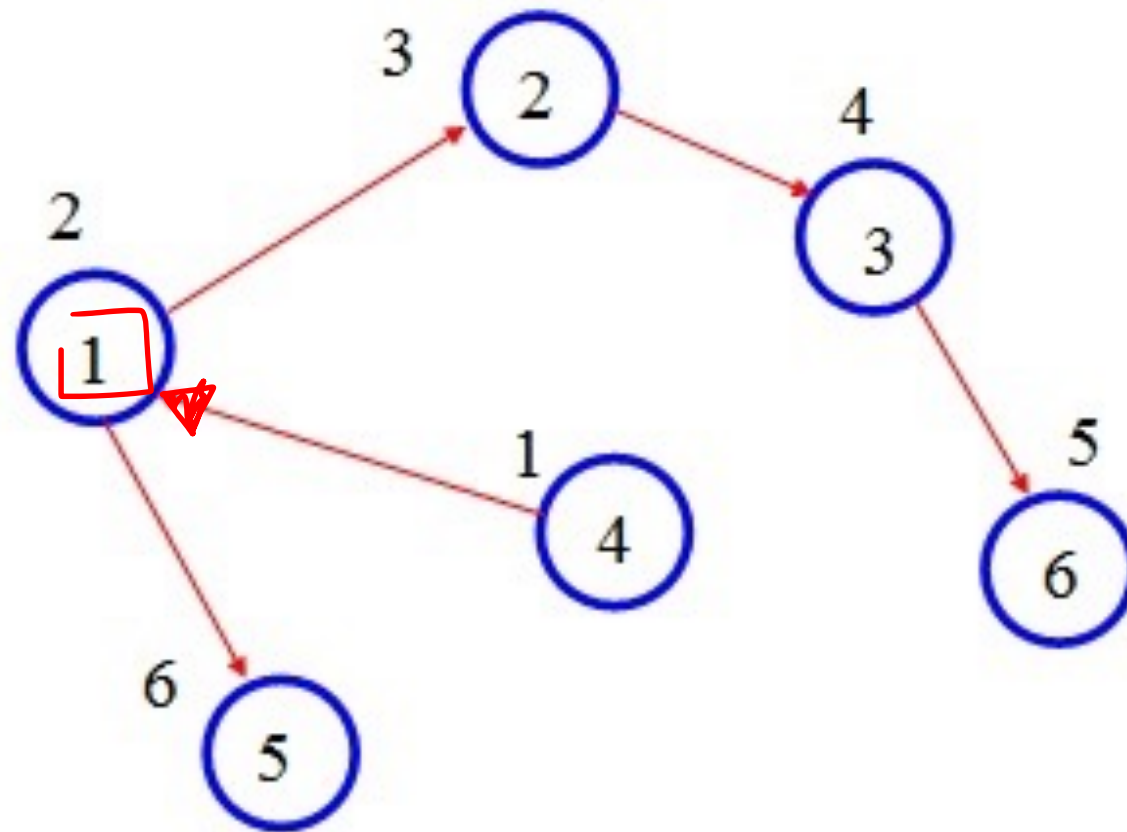
Пример



Пример

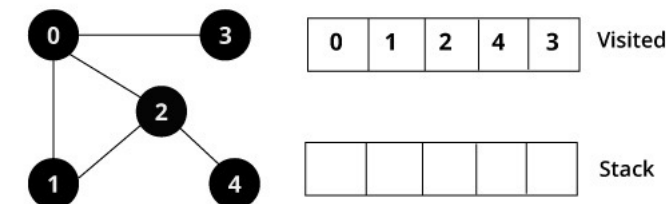
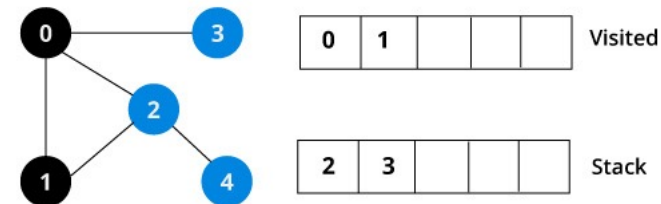
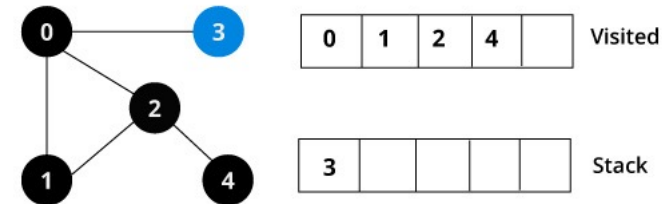
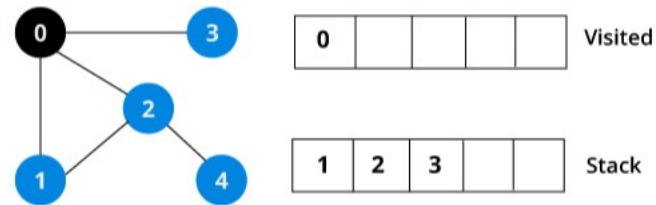
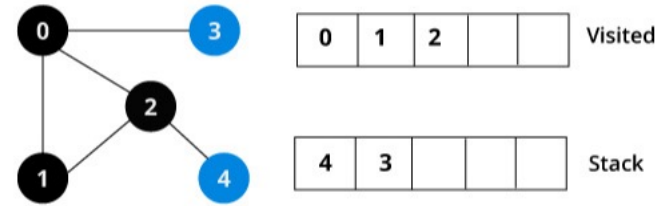
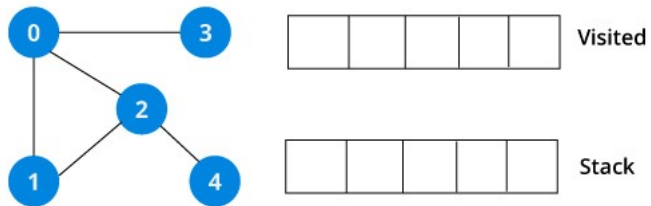


1. $map(1)$ 1 2 3 6 5
2. $map(1)$ 1 2 5 3 6



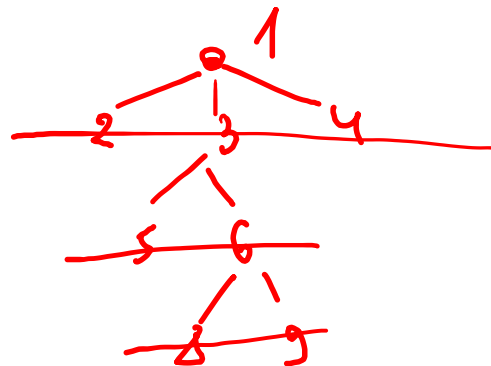
Пример работы не рекурсивной реализации

0 1 2 4 3



Поиск в ширину

Идея *поиска в ширину* заключается в том, что сначала исследуются все вершины, смежные с начальной вершиной (*вершина* с которой начинается обход). Эти вершины находятся на расстоянии 1 от начальной. Затем исследуются все вершины на расстоянии 2 от начальной, затем все на расстоянии 3 и т.д. Обратим внимание, что при этом для каждой вершины сразу находятся *длина* кратчайшего маршрута от начальной вершины.



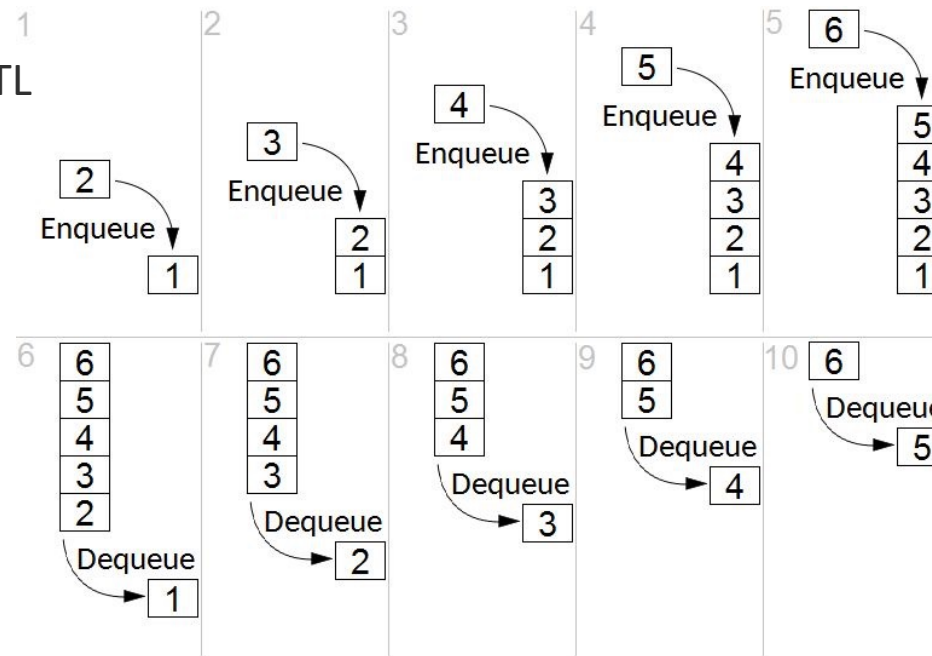
Не рекурсивная реализация

- Все вершины белые
- Выбираем $v \in V$
- Помечаем v черным и помещаем в некоторый контейнер T (очередь)
- Пока T не пусто:
 - Извлекаем v из T
 - Производим некоторые действия над v (зависит от задачи)
 - $\forall v' \in \Gamma(v)$:
 - Если v' - белая, то помечаем v' черным и помещаем в T

Queue

- **Очередью** (англ. – queue) называется структура данных, из которой удаляется первым тот элемент, который был первым в очередь добавлен. То есть очередь в программировании соответствует «бытовому» понятию очереди. Очередь также называют структурой типа FIFO (first in, first out — первым пришел, первым ушел).

В C++ уже есть готовый STL контейнер — queue.



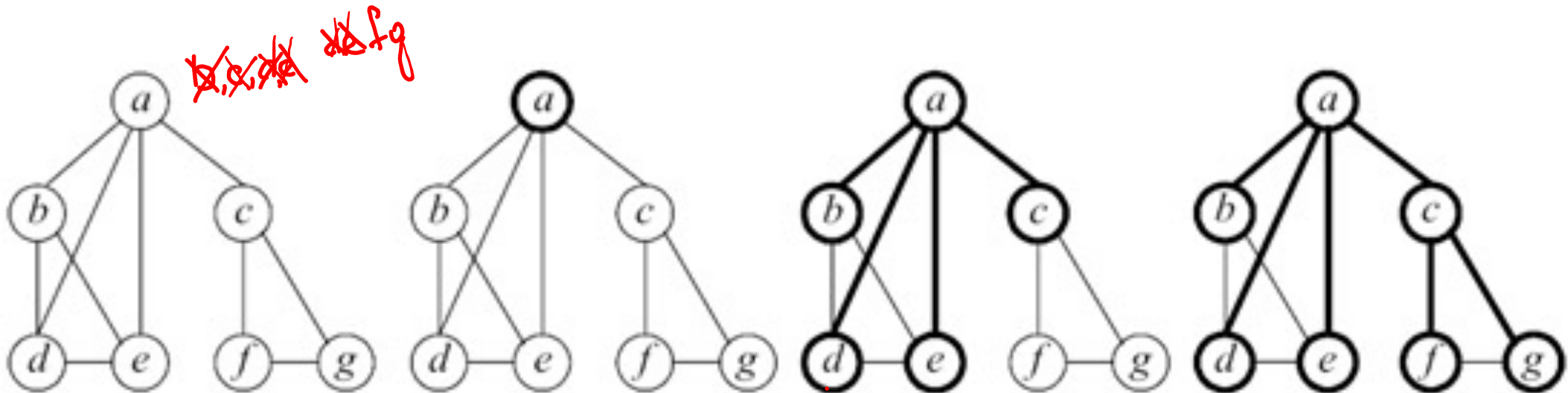
<queue>

- Создание queue <тип данных> <имя>;
- Добавить элемент push(); O(1)
- Удалить первый элемент pop(); O(1)
- Обратиться к первому элементу очереди front(); O(1)
- Обратиться к последнему элементу в очереди back(); O(1)
- Пуста ли очередь empty(); O(1)

Queue C#

```
using System;
using System.Collections.Generic;
namespace Collections
{
    class Program
    {
        static void Main(string[] args)
        {
            Queue<int> numbers = new Queue<int>();
            numbers.Enqueue(3); // очередь 3
            numbers.Enqueue(5); // очередь 3, 5
            numbers.Enqueue(8); // очередь 3, 5, 8
            int queueElement = numbers.Dequeue();
            Console.WriteLine(queueElement);
        }
    }
}
```

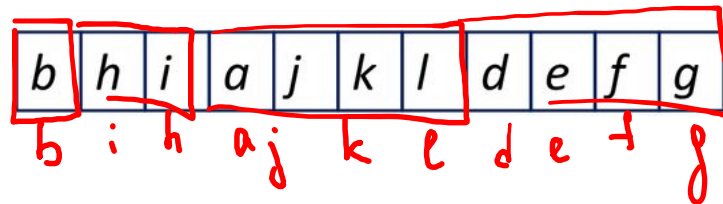
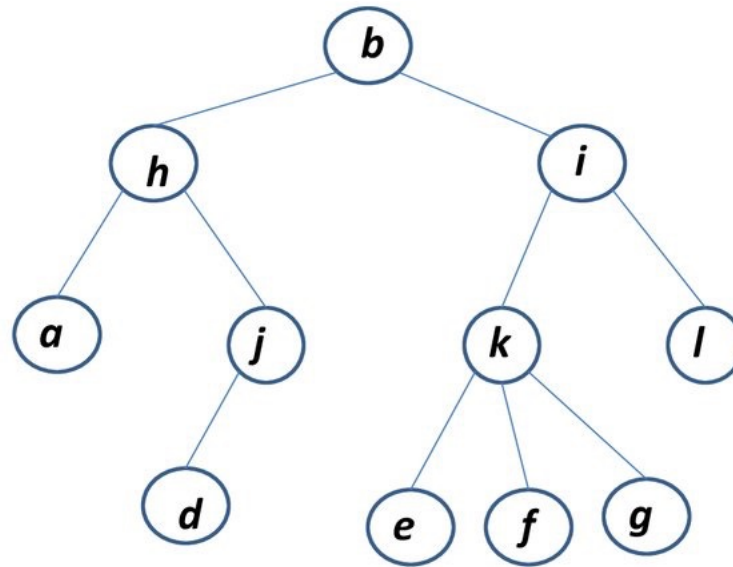
Пример



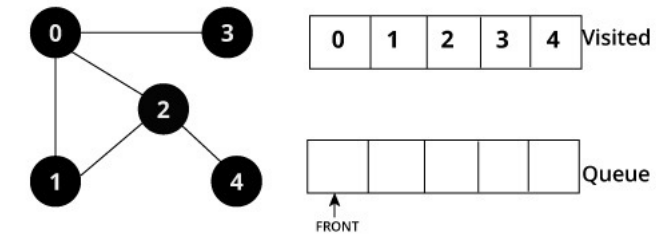
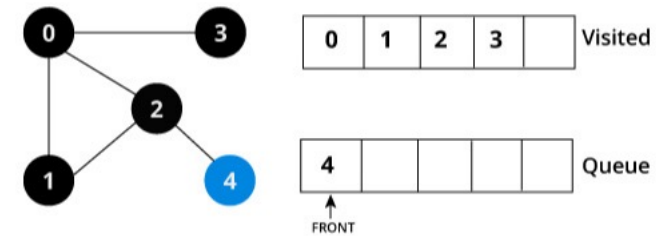
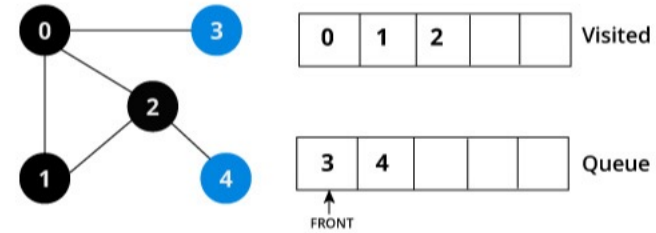
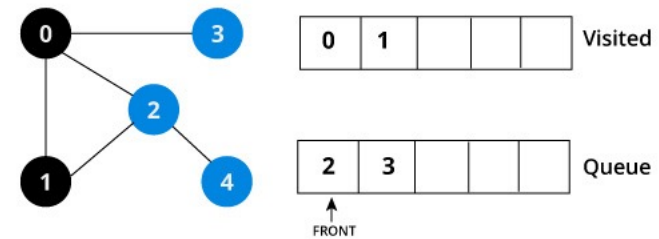
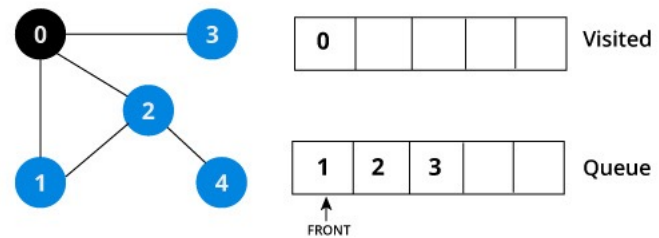
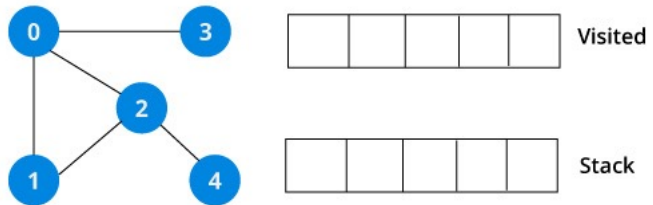

```
queue<int> q; // Очередь вершин
q.push(s); // Добавляем стартовую вершину
vector<bool> used(n); // Вектор признака посещенности вершин
vector<int> d(n), p(n); // Векторы расстояний и предков
used[s] = true; // Стартовую вершину считаем посещенной
p[s] = -1; // У стартовой вершины нет предка
while (!q.empty()) // Пока очередь не пуста
{
    int v = q.front(); // Извлекаем из головы очереди вершину
    q.pop(); // Удаляем извлеченную вершину
    for (auto to : g[v]) // Просмотр всех смежных вершин
    {
int to = g[v][i];
        if (!used[to]) // Если вершина не посещена,
        {
            used[to] = true; // посещаем ее
            q.push(to); // и добавляем к концу очереди
d[to] = d[v] + 1; // Считаем расстояние до вершины
p[to] = v; // Запоминаем предка
        }
    }
}
```

Пример

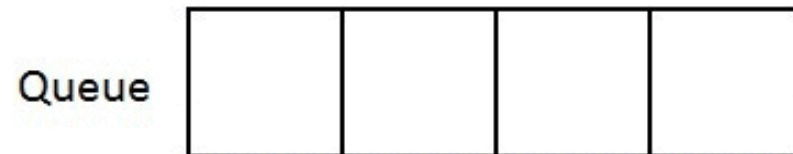
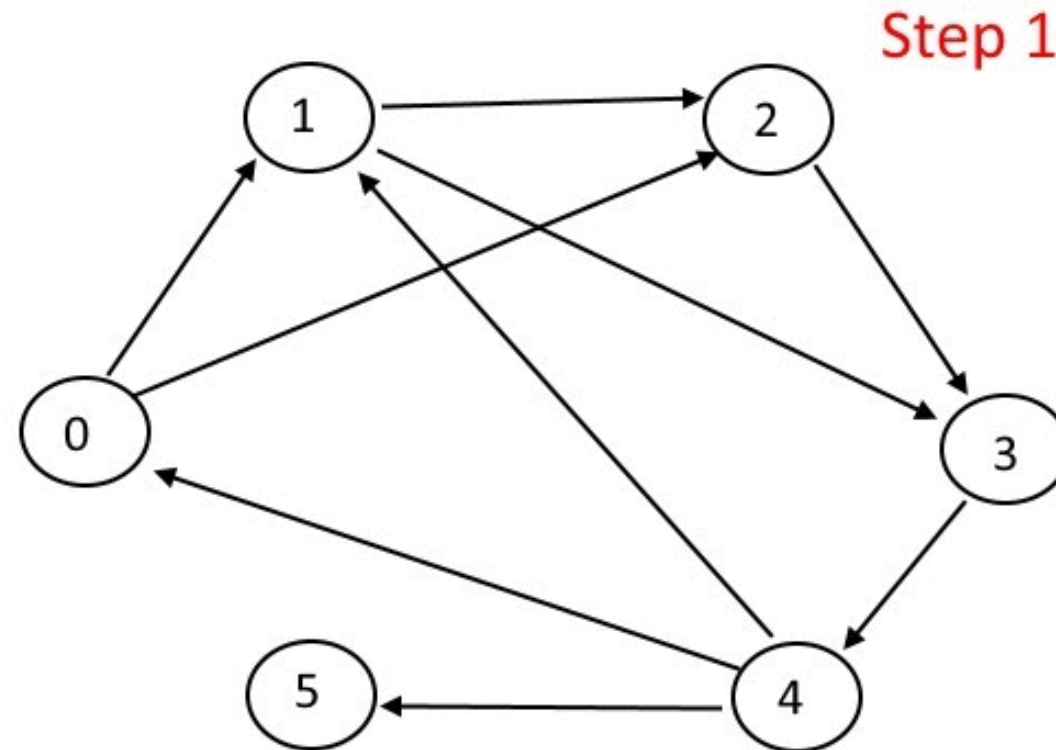
d, h a b i k l e f g



Пример



Пример



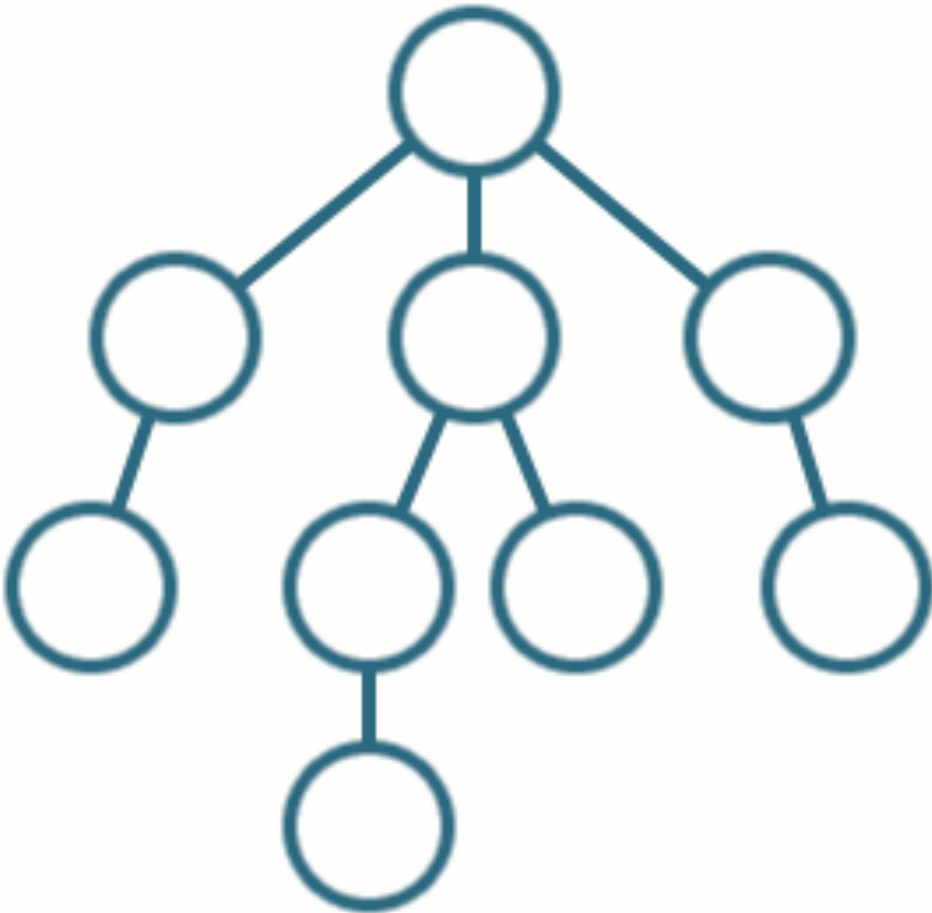
BFS

Применение

Поиск в ширину может применяться для решения задач, связанных с теорией графов:

- Волновой алгоритм поиска пути в лабиринте
- Волновая трассировка печатных плат
- Поиск компонент связности в графе
- Поиск кратчайшего пути между двумя узлами невзвешенного графа
- Поиск в пространстве состояний: нахождение решения задачи с наименьшим числом ходов, если каждое состояние системы можно представить вершиной графа, а переходы из одного состояния в другое — рёбрами графа
- Нахождение кратчайшего цикла в ориентированном невзвешенном графе
- Нахождение всех вершин и рёбер, лежащих на каком-либо кратчайшем пути между двумя вершинами
- Поиск увеличивающего пути в алгоритме Форда-Фалкерсона (алгоритм Эдмондса-Карпа)

DFS



BFS

